A1 Threads

Build_release.sh builds the program
Main takes named argos
Profile_gperf.sh is used to profile program

Summary: The highest impact optimizations from my A.0 implementation did not come from parallelization, but rather from minimizing SYSCALLs and layers of c++ indirection. The final version operated directly on a mmap file with no std i/o or string handling. This brought single threaded execution to ~30 seconds. That being said, doubling threads did result in exponential decay until it asymptotically leveled off at ~8 threads at ~5 seconds.

The optimizations were targeted using gperf (googles rootless perf, what is available on Hellbender).

Using the empirical results below and amdahls law. The sequential portion of the program is about 14%.

Thread search timings

```
sys     0m3.727s
[dzjxb2@g003 A1.simple]$ time ./main --input $(cat path.txt) --threads 2
Number of CPUs: 64
Number of threads (user-specified): 2
Aggregation complete. Output written to: output.csv
Timing report written to: timing.txt

real    0m16.896s
user    0m30.177s
sys     0m3.236s
[dzjxb2@g003 A1.simple]$ time ./main --input $(cat path.txt) --threads 4
Number of CPUs: 64
Number of threads (user-specified): 4
Aggregation complete. Output written to: output.csv
Timing report written to: timing.txt

real    0m8.726s
user    0m29.869s
sys     0m3.327s
[dzjxb2@g003 A1.simple]$ time ./main --input $(cat path.txt) --threads 8
Number of CPUs: 64
Number of threads (user-specified): 8
Aggregation complete. Output written to: output.csv
Timing report written to: timing.txt

real    0m4.773s
user    0m29.491s
sys     0m3.730s
[dzjxb2@g003 A1.simple]$ time ./main --input $(cat path.txt) --threads 16
Number of CPUs: 64
Number of threads (user-specified): 16
Aggregation complete. Output written to: output.csv
Timing report written to: timing.txt

real    0m4.789s
user    0m29.731s
sys     0m3.539s
[dzjxb2@g003 A1.simple]$ time ./main --input $(cat path.txt) --threads 1
Number of CPUs: 64
Number of threads (user-specified): 1
Aggregation complete. Output written to: output.csv
Timing report written to: timing.txt

real    0m33.107s
user    0m30.067s
sys     0m2.978s
[dzjxb2@g003 A1.simple]$
```

Results

```
hcpcs_code,total_paid
T1019,122739547514.26
T1015,49152668633.44
T2016,34904936746.48
99213,33002827263.79
S5125,31342185586.04
99214,29913857937.71
99284,20151783534.42
H2016,19747791724.79
99283,16874996865.61
H2015,16470573872.77
99285,15097200452.31
90837,12067590643.09
S5102,9340779714.30
90834,8821622005.82
T2021,8653524974.89
T2017,8540416258.81
T1017,8423668162.81
T1020,8212560786.12
90999,7739248802.63
A0427,7674558268.41
92507,7484161290.33
H2019,7466162701.30
T2033,7422419481.58
```

Expanded timing info for 8 threads
worker _0 is main

```
=== Timing Report ===
Thread/Task                    Wall Time                              CPU Time
-----------------------------------------------------------------------------------------------
worker_0                       4616.74 ms (4.617 s)                   33253.19 ms (33.253 s)
worker_1                       4176.30 ms (4.176 s)                   32644.01 ms (32.644 s)
worker_2                       4072.30 ms (4.072 s)                   32029.50 ms (32.029 s)
worker_3                       4040.79 ms (4.041 s)                   31808.25 ms (31.808 s)
worker_4                       4187.98 ms (4.188 s)                   32684.84 ms (32.685 s)
worker_5                       4167.49 ms (4.167 s)                   32601.19 ms (32.601 s)
worker_6                       4158.74 ms (4.159 s)                   32553.52 ms (32.554 s)
worker_7                       4346.40 ms (4.346 s)                   32981.68 ms (32.982 s)
```

Gprofile:

```
[dzjxb2@g003 A1.simple]$ pprof --text ./main_gperf ./main.prof.slurmid-0
File: main_gperf
Type: cpu
Showing nodes accounting for 32.25s, 97.46% of 33.09s total
Dropped 30 nodes (cum <= 0.17s)
      flat  flat%   sum%        cum   cum%
     9.20s 27.80% 27.80%     15.98s 48.29%  parse_medicate_record
     7.40s 22.36% 50.17%     32.96s 99.61%  process_region_lines (inline)
     4.50s 13.60% 63.77%      7.41s 22.39%  std::_Hashtable::_M_find_before_node (inline)
     1.39s  4.20% 67.97%      3.89s 11.76%  from_chars_advanced (inline)
     1.34s  4.05% 72.02%      1.70s  5.14%  parse_number_string (inline)
     1.29s  3.90% 75.91%      4.38s 13.24%  std::from_chars (partial-inline)
     1.07s  3.23% 79.15%      1.07s  3.23%  std::__detail::_Hashtable_base::_S_equals (inline)
     0.88s  2.66% 81.81%      0.88s  2.66%  __memcmp_evex_movbe
     0.84s  2.54% 84.35%      1.38s  4.17%  std::_Hash_bytes
     0.60s  1.81% 86.16%      1.14s  3.45%  std::__cxx11::basic_string::_M_replace
     0.48s  1.45% 87.61%      0.48s  1.45%  load_bytes (inline)
     0.43s  1.30% 88.91%      0.43s  1.30%  [main_gperf]
     0.37s  1.12% 90.03%      0.37s  1.12%  __memmove_evex_unaligned_erms
     0.33s     1% 91.02%         1s  3.02%  std::__cxx11::basic_string::basic_string (inline)
     0.33s     1% 92.02%      0.33s     1%  std::__detail::_Mod_range_hashing::operator() (inline)
     0.32s  0.97% 92.99%      1.57s  4.74%  std::operator== (inline)
     0.27s  0.82% 93.80%      9.36s 28.29%  std::__detail::_Map_base::operator[]
     0.24s  0.73% 94.53%      0.24s  0.73%  rounds_to_nearest (inline)
     0.19s  0.57% 95.10%      0.75s  2.27%  std::char_traits::copy (inline)
     0.17s  0.51% 95.62%      0.17s  0.51%  std::__cxx11::basic_string::size (inline)
     0.15s  0.45% 96.07%      9.51s 28.74%  std::unordered_map::operator[] (inline)
     0.14s  0.42% 96.49%      0.52s  1.57%  std::__cxx11::basic_string::_M_construct
     0.12s  0.36% 96.86%      1.56s  4.71%  std::_Hash_impl::hash (inline)
     0.11s  0.33% 97.19%      1.08s  3.26%  std::char_traits::compare (inline)
     0.06s  0.18% 97.37%      7.47s 22.57%  std::_Hashtable::_M_find_node (inline)
     0.03s 0.091% 97.46%      1.26s  3.81%  std::__cxx11::basic_string::assign (inline)
         0     0% 97.46%      5.49s 16.59%  GOMP_parallel
         0     0% 97.46%     27.54s 83.23%  __GI___clone
         0     0% 97.46%      5.55s 16.77%  __libc_start_main
         0     0% 97.46%      5.55s 16.77%  _start
         0     0% 97.46%      4.12s 12.45%  from_chars (inline)
         0     0% 97.46%     27.54s 83.23%  gomp_thread_start
         0     0% 97.46%      5.55s 16.77%  main
         0     0% 97.46%     32.96s 99.61%  main._omp_fn.0
         0     0% 97.46%     27.54s 83.23%  start_thread
         0     0% 97.46%      0.33s     1%  std::_Hashtable::_M_bucket_index (inline)
         0     0% 97.46%      0.17s  0.51%  std::__cxx11::basic_string::_M_set_length (inline)
         0     0% 97.46%      0.75s  2.27%  std::__cxx11::basic_string::_S_copy (inline)
         0     0% 97.46%      0.35s  1.06%  std::__cxx11::basic_string::_S_copy_chars (inline)
         0     0% 97.46%      0.33s     1%  std::__detail::_Hash_code_base::_M_bucket_index (inline)
         0     0% 97.46%      1.56s  4.71%  std::__detail::_Hash_code_base::_M_hash_code (inline)
         0     0% 97.46%      2.64s  7.98%  std::__detail::_Hashtable_base::_M_equals (inline)
         0     0% 97.46%      1.57s  4.74%  std::__detail::_Hashtable_base::_M_key_equals (inline)
         0     0% 97.46%      1.56s  4.71%  std::__str_hash_base::operator() (inline)
         0     0% 97.46%      1.57s  4.74%  std::equal_to::operator() (inline)
[dzjxb2@g003 A1.simple]$
```